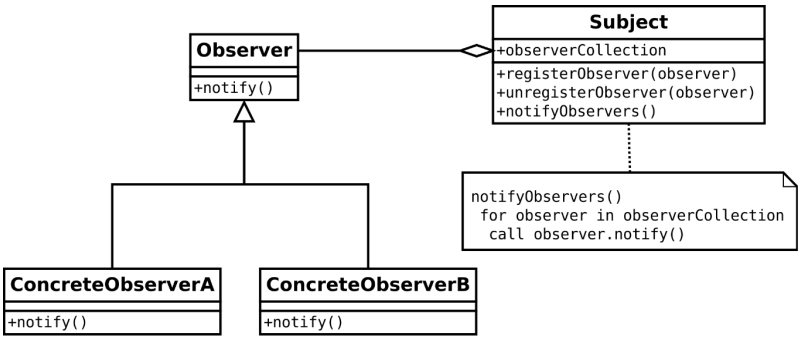
Fiche résumé : pattern Observer (Observateur)

Pattern comportemental du GoF, aussi appelé Publish-Subscribe.

Problématique et intention

- Problématique : comment prévenir automatiquement plusieurs objets (par exemple des affichages) d'un changement d'état (par exemple la température d'une station météo) sans créer de dépendances directes entre eux ?
- Intention : prévenir d'autres objets d'un changement d'état sans avoir besoin de connaître leur fonctionnement interne.

Diagramme de classes générique



Rôles des classes

- Subject : interface qui définit inscrire, désinscrire et notifier (Sujet).
- ConcreteSubject : possède l'état observé et la liste des observateurs (StationMeteo).
- Observer : interface qui définit ce que fait un observateur lorsqu'il est notifié (Observateur).
- ConcreteObserver : définit sa réaction à la notification (AffichageTemperature, AffichageStatistiques).
- Déroulé : le capteur appelle modifierTemperature(), la station met à jour son état, appelle notifierObservateurs(), puis actualiser() sur chaque observateur inscrit.

Lien avec SOLID

- SRP : la station gère l'état et la liste, chaque affichage gère son rendu.
- OCP : ajouter un affichage = une nouvelle classe et un ajouterObservateur, sans modifier la station.
- LSP : un observateur peut en remplacer un autre sans rien casser.
- ISP : l'interface Observateur n'a qu'une méthode.
- DIP : la station ne dépend que de l'interface Observateur, pas des classes concrètes.

Push et pull

- Push : le sujet envoie la donnée, observateur.actualiser(temperature). Avantages : simple, faible couplage, un seul appel. Inconvénients : données inutiles, une nouvelle donnée oblige à modifier la signature partout.
- Pull : le sujet s'envoie lui-même, observateur.actualiser(this), et l'observateur appelle les getters. Avantages : chacun lit ce dont il a besoin, la signature ne change pas. Inconvénients : l'observateur dépend du sujet, plus d'appels, l'état peut changer entre la notification et la lecture.
- Push si les besoins des observateurs sont les mêmes, pull s'ils sont variés.

Limites

- Fuite mémoire : un observateur jamais désinscrit reste référencé par le sujet.
- Ordre de notification non garanti.
- Notifications en cascade : risque de boucle infinie.
- Exception dans un observateur : les suivants ne sont pas prévenus.
- Multithread : désinscription pendant une notification = erreur.
- Mises à jour inutiles : en push, des observateurs reçoivent des données qui ne les concernent pas.

Observer et Observable dans le SDK Java (dépréciés depuis Java 9)

- Observable est une classe : le sujet en hérite (addObserver, setChanged, notifyObservers). Observer est une interface avec update(Observable o, Object arg).
- Défauts : héritage imposé, Object arg non typé (cast nécessaire), peu adapté au multithread.
- Alternatives : PropertyChangeListener, Flow, ou sa propre interface.

Sources

Wikipédia, Observateur (patron de conception) ; Gamma, Helm, Johnson, Vlissides, Design Patterns (1994) ; documentation Oracle, java.util.Observable.